Vulnerability Management Policy

Transparency Hub Network Version: 1.0 **Effective Date:** March 5, 2026 **Last Updated:** March 5, 2026 **Owner:** Engineering Team **Classification:** Internal

---

1. Purpose

This policy establishes the processes, responsibilities, and standards for identifying, assessing, prioritizing, remediating, and tracking security vulnerabilities across all Transparency Hub Network systems, applications, and infrastructure. The goal is to minimize the window of exposure to known vulnerabilities and maintain a strong security posture that protects our users, their data, and our platform.

---

2. Scope

This policy applies to:

- All application source code (frontend, backend, mobile)
- Third-party libraries and dependencies
- Cloud infrastructure (Azure Functions, Azure Static Web Apps, Azure Blob Storage, PostgreSQL)
- Third-party integrations and APIs (Attendee.dev, Stripe, Paystack, Google OAuth, Zoom, Microsoft Teams)
- Development, staging, and production environments
- All personnel involved in development, operations, and security

Repositories Covered:

Repository	Type	Language
`Transparency-Hub/Staticwebsite-App`	Admin Dashboard	TypeScript / Next.js
`Transparency-Hub/AzureFunction`	Backend API	Python / Azure Functions
`Transparency-Hub/Staticwebsite`	Public Website	TypeScript / Next.js
`Transparency-Hub/docs`	Documentation	Mintlify / Markdown

---

3. Vulnerability Sources

Vulnerabilities are identified through the following channels:

3.1 Automated Detection

Source	Type	Frequency	Coverage
GitHub CodeQL (SAST)	Static code analysis	Every PR, push to main, weekly	Python, JavaScript, TypeScript
OWASP ZAP (DAST)	Dynamic application scanning	Weekly, on-demand	Frontend + Backend API (staging)
GitHub Dependabot (SCA)	Dependency vulnerability scanning	Continuous	All repositories

npm audit / pip audit	Package vulnerability scanning	On-demand, CI pipeline	Node.js and Python packages

3.2 Manual Detection

Source	Description
Code Reviews	Security-focused peer review on every PR
Penetration Testing	Periodic third-party security assessments
Bug Bounty / Responsible Disclosure	External security researchers (see Section 11)
Internal Security Reviews	Quarterly review of architecture and access controls
Incident Analysis	Post-incident reviews identifying root cause vulnerabilities

3.3 External Intelligence

Source	Description
CVE Databases	National Vulnerability Database (NVD), MITRE CVE
Vendor Advisories	Azure, GitHub, Zoom, Google, Stripe security bulletins
OWASP Updates	OWASP Top 10 and emerging threat intelligence
Community Alerts	Security mailing lists, social media, developer communities

---

4. Vulnerability Classification

4.1 Severity Ratings

We use the **Common Vulnerability Scoring System (CVSS) v3.1** as the primary severity framework, supplemented by contextual risk assessment.

Severity	CVSS Score	Description	Examples
Critical	9.0 – 10.0	Actively exploitable, immediate risk of data breach or system compromise	Remote code execution, SQL injection with data access, authentication bypass
High	7.0 – 8.9	Significant vulnerability with high potential impact	Privilege escalation, stored XSS with session theft, SSRF with internal access
Medium	4.0 – 6.9	Moderate vulnerability with limited impact or requiring specific conditions	Reflected XSS, information disclosure, CSRF on non-critical actions

Low	0.1 – 3.9	Minor vulnerability with minimal impact	Missing non-critical headers, verbose error messages, clickjacking on public pages
Informational	0.0	Best practice recommendation, no direct security impact	Code quality improvements, deprecated API usage

4.2 Contextual Risk Factors

CVSS scores are adjusted based on:

- **Data sensitivity:** Vulnerabilities affecting payment data, PII, or authentication are escalated
- **Exposure:** Internet-facing vs internal-only components
- **Exploitability:** Proof-of-concept availability, exploit complexity
- **Business impact:** Effect on core functionality (membership management, payments, meeting recording)
- **Affected user base:** Number of associations and members potentially impacted

---

5. Remediation Timelines

5.1 Service Level Agreements (SLAs)

Severity	Acknowledgement	Remediation Deadline	Verification
Critical	Within 4 hours	24 hours (emergency patch)	Same day
High	Within 8 hours	7 calendar days	Within 2 days of fix
Medium	Within 24 hours	30 calendar days	Within 5 days of fix
Low	Within 48 hours	90 calendar days (next quarter)	Next scheduled scan
Informational	Next sprint planning	Best effort / next major release	Next scheduled scan

5.2 Emergency Response (Critical Vulnerabilities)

When a critical vulnerability is identified:

1. **Immediate Assessment** (0–4 hours)
    - Confirm the vulnerability and assess active exploitation
    - Determine scope of affected systems and data
    - Notify the engineering lead
2. **Containment** (4–8 hours)
    - Apply temporary mitigations (WAF rules, feature flags, access restrictions)
    - Isolate affected systems if necessary
    - Begin developing a permanent fix
3. **Remediation** (8–24 hours)
    - Deploy the permanent fix to staging, run SAST/DAST verification
    - Deploy to production with expedited review
    - Monitor for recurrence

4. **Post-Incident** (24–72 hours)
    - Conduct root cause analysis
    - Document lessons learned
    - Update detection rules to prevent similar vulnerabilities
    - Notify affected users if data exposure occurred (see Section 9)

5.3 Exceptions and Deferrals

If a vulnerability cannot be remediated within the SLA:

- A **risk acceptance** must be documented with justification
- Compensating controls must be identified and implemented
- The exception must be approved by the engineering lead
- Maximum deferral: one additional SLA period (e.g., High deferred from 7 to 14 days)
- Critical vulnerabilities cannot be deferred

---

6. Vulnerability Lifecycle

```
┌──────────────┐    ┌──────────────┐    ┌──────────────┐
│  Detection   │──▶│   Triage     │──▶│   Analysis   │
│              │    │              │    │              │
│ • CodeQL     │    │ • Severity   │    │ • Root cause │
│ • ZAP        │    │ • Priority   │    │ • Impact     │
│ • Dependabot │    │ • Assignment │    │ • Scope      │
│ • Manual     │    │              │    │              │
└──────────────┘    └──────────────┘    └──────────────┘
                                              │
                                              ▼
┌──────────────┐    ┌──────────────┐    ┌──────────────┐
│   Closed     │◀──│ Verification │◀──│ Remediation  │
│              │    │              │    │              │
│ • Resolved   │    │ • Re-scan    │    │ • Fix code   │
│ • False pos. │    │ • Pen test   │    │ • Update dep │
│ • Accepted   │    │ • Code review│    │ • Config     │
└──────────────┘    └──────────────┘    └──────────────┘
```

6.1 Detection

- Automated tools run continuously and on schedule (see Section 3.1)
- Findings are automatically created as GitHub Security Alerts or Issues
- Manual findings are logged as GitHub Issues with the `security` label

6.2 Triage

- All new findings are triaged within the acknowledgement SLA
- Each finding is assigned:
    - **Severity** (Critical / High / Medium / Low / Informational)
    - **Owner** (responsible engineer)
    - **Status** (Open / In Progress / Resolved / Won't Fix / False Positive)
- False positives are documented with justification and suppressed in tool configuration

6.3 Analysis

- Root cause identified (coding error, misconfiguration, vulnerable dependency, design flaw)
- Impact assessment completed (what data/systems are at risk)
- Attack vector documented (how could this be exploited)

6.4 Remediation

Remediation approaches in order of preference:

1. **Fix the code** — Eliminate the vulnerability at the source
2. **Update the dependency** — Upgrade to a patched version
3. **Apply a workaround** — Temporary mitigation (WAF rule, input filter, feature disable)
4. **Accept the risk** — Documented exception with compensating controls (last resort)

6.5 Verification

- SAST re-scan confirms code-level fixes
- DAST re-scan confirms runtime fixes
- Dependency audit confirms package updates
- Peer review confirms fix quality and completeness

6.6 Closure

- Vulnerability marked as resolved with:
  - Fix commit reference
  - Verification scan results
  - Date of resolution
  - Time-to-remediate metric recorded

---

7. Dependency Management

7.1 Monitoring

- **GitHub Dependabot** monitors all repositories continuously
- Alerts created automatically when vulnerabilities are published in dependency trees
- Both direct and transitive dependencies are covered

7.2 Patching Policy

Severity	Action	Timeline
Critical	Immediate update, emergency release if needed	24 hours
High	Update in next release cycle	7 days
Medium	Update in next sprint	30 days
Low	Bundle with next planned update	90 days

7.3 Transitive Dependencies

When a vulnerability exists in a transitive (indirect) dependency and no upstream fix is available:

- Use `npm overrides` (Node.js) or dependency pinning (Python) to force a safe version
- Document the override with justification

- Monitor for upstream fixes and remove overrides when available
- If no safe version exists, evaluate alternative packages

7.4 End-of-Life Dependencies

- Dependencies that are no longer maintained are flagged for replacement
- Migration plans are created within 30 days of EOL identification
- No new features should be built on EOL dependencies

---

8. Infrastructure Vulnerability Management

8.1 Azure Platform

- Azure platform patches are managed by Microsoft (PaaS model)
- Azure Functions runtime updates are monitored and applied within 14 days
- Azure Security Center recommendations are reviewed monthly

8.2 Database

- PostgreSQL security patches applied within 7 days of release
- Database access audited quarterly
- Connection credentials rotated every 90 days

8.3 Configuration Security

- Azure resource configurations reviewed quarterly against CIS benchmarks
- TLS 1.2+ enforced on all endpoints
- Network access rules reviewed monthly
- Storage account access policies reviewed quarterly

---

9. Disclosure and Communication

9.1 Internal Communication

- Critical and High vulnerabilities are communicated to the engineering team immediately via direct message
- Weekly security summary shared during team standup
- Monthly vulnerability metrics included in engineering reports

9.2 User Notification

Users (association administrators and members) are notified when:

- A vulnerability resulted in confirmed or suspected data exposure
- A vulnerability required users to take action (e.g., password reset, session invalidation)
- Notification is required by applicable data protection regulations (NDPR, GDPR)

Notification timeline: Within **72 hours** of confirming user data impact.

9.3 Regulatory Notification

Where required by law (e.g., NDPR), the relevant data protection authority is notified within the legally mandated timeframe.

---

10. Metrics and Reporting

10.1 Key Metrics

Metric	Target	Measurement
Mean Time to Detect (MTTD)	< 24 hours	Time from vulnerability introduction to detection
Mean Time to Remediate (MTTR)	Critical: < 24h, High: < 7d	Time from detection to verified fix
SLA Compliance Rate	> 95%	Percentage of vulnerabilities fixed within SLA
Open Vulnerability Count	Trending downward	Total open findings by severity
False Positive Rate	< 15%	Percentage of findings classified as false positive
Dependency Currency	> 90% up to date	Percentage of dependencies on latest secure version

10.2 Reporting Cadence

Report	Audience	Frequency
Vulnerability Dashboard	Engineering Team	Real-time (GitHub Security tab)
Weekly Summary	Engineering Lead	Weekly
Monthly Security Report	Leadership	Monthly
Quarterly Review	Full Team	Quarterly

---

11. Responsible Disclosure

11.1 Reporting Channel

Security researchers and external parties can report vulnerabilities to:

Email: security@transparencyhubnetwork.ai

11.2 Our Commitments

- **Acknowledgement** within 48 hours of receiving a report
- **Initial assessment** within 5 business days
- **Status update** within 10 business days
- **Fix timeline** communicated within 14 business days
- **Credit** given to reporters (with permission) in security advisories

11.3 Safe Harbor

We will not pursue legal action against researchers who:

- Act in good faith to identify and report vulnerabilities
- Do not access, modify, or delete user data beyond what is necessary to demonstrate the vulnerability
- Do not disrupt or degrade our services

- Allow reasonable time for remediation before public disclosure
- Do not exploit the vulnerability beyond proof of concept

---

12. Roles and Responsibilities

Role	Responsibilities
Engineering Lead	Overall policy ownership, exception approvals, escalation point, monthly reporting
Developers	Remediate assigned vulnerabilities within SLA, follow secure coding standards, review CodeQL alerts on their PRs
DevOps / Infrastructure	Monitor infrastructure vulnerabilities, apply platform patches, manage secrets rotation
All Engineers	Report suspected vulnerabilities, participate in security reviews, follow this policy

---

13. Policy Review

This policy is reviewed and updated:

- **Quarterly** as part of the regular security review cycle
- **Immediately** following a significant security incident
- **When** new tools, technologies, or compliance requirements are adopted

---

14. Related Documents

Document	Description
SSDLC	Secure Software Development Lifecycle — full development security practices
SAST & DAST	Detailed security testing documentation — tools, coverage, and configuration
Privacy Policy	User data protection and privacy practices
Terms of Service	User agreement and acceptable use

---

15. Document Control

Version	Date	Author	Changes
1.0	2026-03-05	Engineering Team	Initial version

For questions about this policy, contact: support@transparencyhubnetwork.ai